A Computer-Assisted Proof That the Autoconvolution Constant C_{1a} Satisfies $C_{1a} \geq \frac{7}{5}$

Jai Bajaj

Abstract

We prove that the autoconvolution constant

$$C_{1a} \;=\; \inf_f \frac{\|f * f\|_{L^\infty}}{\left(\int f\right)^2},$$

taken over nonnegative functions supported on $(-\frac{1}{4}, \frac{1}{4})$, satisfies $C_{1a} \geq \frac{7}{5}$. The argument starts from the Cloninger–Steinerberger reduction [1] from continuous functions to windowed sums of bin masses, but is organized around the exact objects used in the accompanying Lean development: a half-open bin partition, a cumulative-floor canonical discretization, and a window-dependent correction term. The finite verification is a dyadic branch-and-prune cascade with terminal parameters $n = 64$, $m = 20$, and $d = 128$, ending with zero survivors. All analytic steps from the continuous problem to the terminal-grid statement are formalized in Lean; the only imported assumption is the finite terminal certificate asserting that every composition in $\mathbb{N}^{128}$ of total mass 20 is eliminated by some admissible window.

Keywords. Autoconvolution, Sidon sets, branch-and-prune, computer-assisted proof, Lean formalization.

1. Introduction

1.1. The Constant C_{1a}

Let

$$\mathcal{F} \;=\; \left\{ f \in L^1(\mathbb{R}) : f \geq 0,\ \operatorname{supp}(f) \subseteq \left(-\tfrac{1}{4}, \tfrac{1}{4}\right),\ \int_{\mathbb{R}} f > 0 \right\}.$$

The *autoconvolution* of f is

$$(f * f)(t) \;=\; \int_{-\infty}^{\infty} f(x)\, f(t-x)\, dx.$$

Since $f \in L^1(\mathbb{R})$ has compact support, $f * f$ is continuous, nonnegative, and supported on $[-\frac{1}{2}, \frac{1}{2}]$.

Definition 1.1. The *autoconvolution ratio* is

$$R(f) \;=\; \frac{\|f * f\|_{L^\infty}}{\left(\int f\right)^2},$$

and the *autoconvolution constant* is

$$C_{1a} \;=\; \inf_{f \in \mathcal{F}} R(f).$$

Bound	Value	Reference
Published upper bound	≤ 1.50992	Matolcsi & Vinuesa [2]
Published lower bound	≥ 1.28	Cloninger & Steinerberger [1]
Lower bound	≥ 1.4	This work

Table 1: Published bounds on C_{1a} in the normalization used here.

Remark 1.2 (Normalization and the formal ratio)**.** By homogeneity, $R(\lambda f) = R(f)$ for every $\lambda > 0$, so one may normalize $\int f = 1$. The Lean definition `autoconvolution_ratio` uses the essential supremum `eLpNorm` of $f * f$; for $f \in \mathcal{F}$ this agrees with the maximum of $f * f$ on $[-\frac{1}{2}, \frac{1}{2}]$ because $f * f$ is continuous.

Why this constant matters. C_{1a} is connected to the asymptotic theory of Sidon and generalized Sidon sets in additive combinatorics [**?**, **?**, **?**, **?**, 1]. A *Sidon set* is a set of integers where all pairwise sums are distinct; equivalently, the "representation function" of the sumset has values at most 2 everywhere except at doubled elements. The constant C_{1a} governs how efficiently mass can be distributed to minimize the peak of the autoconvolution — the continuous analog of controlling pairwise-sum collisions.

1.2. Known Bounds

The *upper bound* comes from exhibiting explicit functions whose autoconvolution peak is small. This is a constructive task: find a good f and compute its peak.

The *lower bound* is fundamentally harder: one must show that *no* function can achieve a peak below the threshold. This requires exhaustive elimination of all possibilities, which is the subject of this document.

The published lower bounds of Martin–O'Bryant [**?**, **?**], Matolcsi–Vinuesa [2], and Cloninger–Steinerberger [1] give the progression

$$1 \;<\; 1.182778 \;<\; 1.262 \;<\; 1.2748 \;<\; 1.28$$

in the present normalization, while the older upper bound $\pi/2$ comes from Schinzel–Schmidt [**?**].

1.3. Proof Strategy at a High Level

The proof has two components: a continuous reduction and a finite certificate. The continuous reduction is formalized in Lean; the finite certificate is produced by an exhaustive branch-and-prune computation.

At a high level, the reduction proceeds in three steps:

(i) **Reduce** the infinite-dimensional optimization over functions to a family of windowed quadratic expressions in bin masses.

(ii) **Discretize** the normalized bin masses by a specific cumulative-floor map, producing a composition $c \in \mathbb{N}^{2n}$ of total mass m together with a quantified per-window error term.

(iii) **Certify** that every terminal composition at $(n, m) = (64, 20)$ is eliminated by some window whose test value exceeds the corrected threshold.

The terminal statement is verified by a dyadic cascade from $d = 4$ to $d = 128$, using reversal symmetry, asymmetry pruning, and the window-dependent correction term. The Lean theorem `autoconvolution_ratio_ge_7_5` combines the analytic reduction with the imported terminal certificate `cascade_all_pruned`.

2. From Functions to Bin Averages

Our first order of business is to replace the infinite-dimensional optimization — over all nonnegative functions — with something finite-dimensional and computable.

2.1. The Partition

Fix a positive integer n and write $d = 2n$. The formalization uses the half-open partition

$$I_i \;=\; \left[-\tfrac{1}{4} + \frac{i}{4n},\, -\tfrac{1}{4} + \frac{i+1}{4n}\right), \qquad i = 0, \ldots, d-1.$$

Each bin has width $\frac{1}{4n}$, and the family $\{I_i\}_{i=0}^{d-1}$ partitions $[-\frac{1}{4}, \frac{1}{4})$.

Figure **??** illustrates the support partition and the induced vectors of bin masses and bin averages.

2.2. Bin Masses and Bin Averages

Assume now that $f \in \mathcal{F}$ and $\int f = 1$. Define the *bin masses*

$$\mu_i \;=\; \int_{I_i} f(x)\, dx \;\geq\; 0, \qquad i = 0, \ldots, d-1,$$

and the associated *bin averages*

$$a_i \;=\; 4n\, \mu_i.$$

Since $|I_i| = \frac{1}{4n}$, the quantity a_i is the average value of f on I_i whenever f is constant on that bin.

Lemma 2.1 (Partition of mass)**.** *For the normalized function f, the bin masses satisfy*

$$\sum_{i=0}^{d-1} \mu_i \;=\; 1, \qquad \sum_{i=0}^{d-1} a_i \;=\; 4n.$$

Proof. The half-open bins partition $[-\frac{1}{4}, \frac{1}{4})$, while $\mathrm{supp}(f) \subseteq (-\frac{1}{4}, \frac{1}{4})$ excludes the endpoints. Thus the sum of the bin integrals is $\int f = 1$, and the second identity is obtained by multiplying by $4n$. □

Remark 2.2 (Formal counterpart)**.** The Lean lemma `sum_bin_masses_eq_one` is exactly the first identity above. It is used repeatedly to normalize the cumulative discretization map introduced in Section 3.

Definition 2.3. The *normalized mass simplex* and the *bin-average simplex* are

$$\Delta_n \;=\; \left\{\mu \in \mathbb{R}^d_{\geq 0} : \sum_{i=0}^{d-1} \mu_i = 1\right\},$$

$$A_n \;=\; \left\{a \in \mathbb{R}^d_{\geq 0} : \sum_{i=0}^{d-1} a_i = 4n\right\} \;=\; 4n\, \Delta_n.$$

The vector $\mu = (\mu_0, \ldots, \mu_{d-1})$ is the object discretized in Lean, whereas the windowed quadratic forms are most naturally written in the rescaled coordinates $a = 4n\, \mu$.

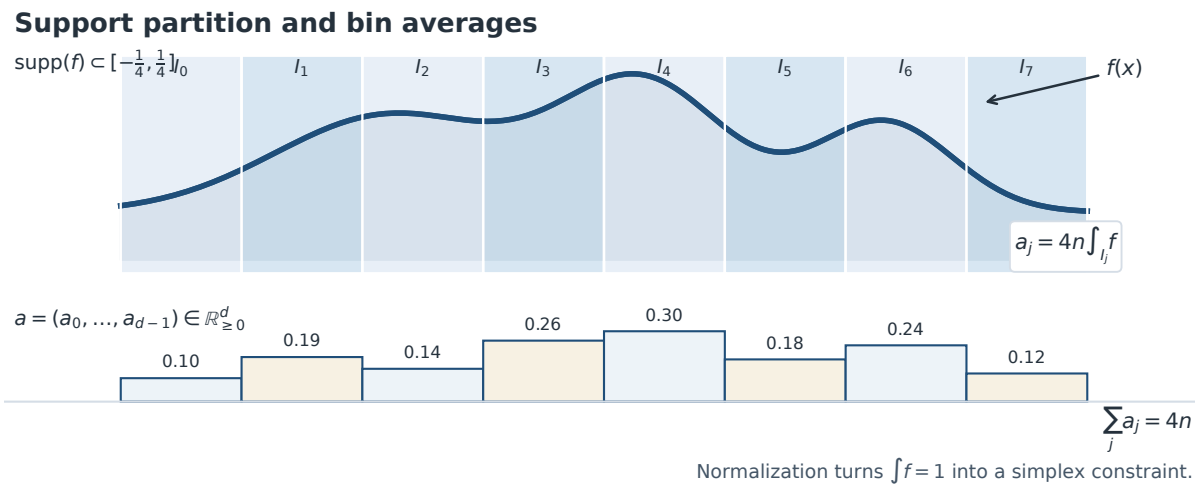

Figure 1: A nonnegative profile supported on $[-\frac{1}{4}, \frac{1}{4}]$, together with the half-open partition into equal bins. The upper panel shows the support decomposition, while the lower panel records the induced averages $a_i = 4n\,\mu_i$ and the simplex identity $\sum_i a_i = 4n$ after normalizing $\int f = 1$.

2.3. Window Lower Bound for the Ratio

The next lemma is the analytic bridge from the continuous problem to the discrete windowed expressions of Cloninger and Steinerberger [1, Lemma 1].

Lemma 2.4 (Per-window test-value bound)**.** *Let $f \in \mathcal{F}$ satisfy $\int f = 1$, and let $a_i = 4n \int_{I_i} f$. Then for every pair of integers $\ell \geq 2$ and $s_{\mathrm{lo}} \geq 0$,*

$$\|f * f\|_{L^\infty} \;\geq\; \frac{1}{4n\ell} \sum_{s=s_{\mathrm{lo}}}^{s_{\mathrm{lo}}+\ell-2} \sum_{\substack{0 \leq i,j \leq d-1 \\ i+j=s}} a_i a_j .$$

Proof sketch. Write $f_i = f\,\mathbf{1}_{I_i}$. Then $f = \sum_i f_i$ and $(f * f)(t) = \sum_{i,j} (f_i * f_j)(t)$. The support of $f_i * f_j$ is contained in $I_i + I_j$, an interval of length $\frac{1}{2n}$ whose location depends only on $i + j$. If one fixes the sum-index window $s_{\mathrm{lo}} \leq i + j \leq s_{\mathrm{lo}} + \ell - 2$, the corresponding Minkowski sums form an interval of total length $\frac{\ell}{4n}$. The total L^1 mass contributed by these terms is

$$\sum_{s=s_{\mathrm{lo}}}^{s_{\mathrm{lo}}+\ell-2} \sum_{i+j=s} \frac{a_i}{4n} \frac{a_j}{4n}.$$

Applying the averaging inequality $\|h\|_{L^\infty} \geq |J|^{-1} \int_J h$ to this interval gives the claimed lower bound. □

Remark 2.5 (Formal counterpart)**.** The Lean theorem `continuous_test_value_le_ratio` is the formal version of Lemma 2.3. Its statement is *per window*: for each admissible (ℓ, s_{lo}) with $\ell \geq 2$, the continuous windowed test value is bounded above by the autoconvolution ratio. The maximized reduction used in the paper is obtained only after taking the maximum over all windows.

Remark 2.6 (Intuition)**.** The products $a_i a_j$ quantify how much mass from bins i and j can contribute to the autoconvolution. Narrow windows carry the favorable prefactor $\frac{1}{4n\ell}$ but capture fewer pairs; wider windows capture more pairs but average over a longer interval. The branch-and-prune computation exploits this competition.

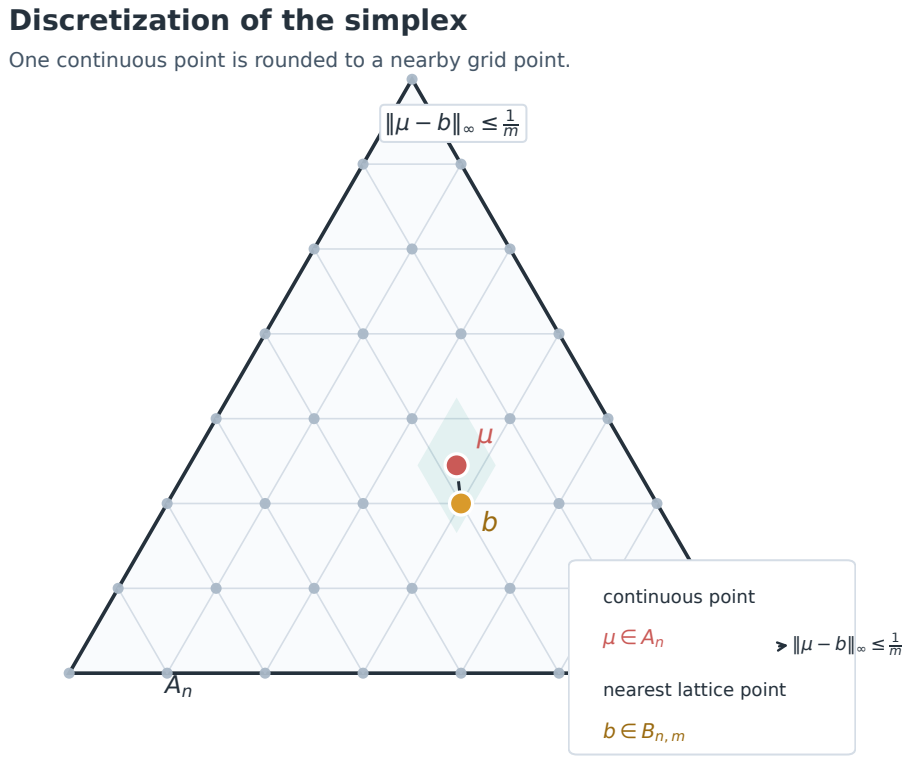

Figure 2: A three-coordinate slice of the normalized simplex. The point μ is replaced by a nearby lattice point $b \in B_{n,m}$; in the formal proof this representative is chosen by the cumulative-floor map described below.

3. Discretization of the Simplex

Having reduced the problem to minimizing over the continuous simplex A_n, we face a new obstacle: A_n is uncountable. To bring the problem within reach of a computer, we discretize the normalized mass simplex Δ_n. Regular simplex grids are standard in finite-dimensional optimization [**?**]; here the key point is that the formal proof uses a particular lattice point selected by a cumulative-floor rule.

3.1. The Lattice

Fix a positive integer m. In normalized mass coordinates, the natural grid has spacing $1/m$.

Definition 3.1. Fix a positive integer m (the "grid resolution"). The *discrete lattice* is

$$B_{n,m} = \left\{ b \in \left(\tfrac{1}{m}\mathbb{N}\right)^d \;\middle|\; \sum_{j=0}^{d-1} b_j = 1 \right\},$$

where each b_j is a nonnegative multiple of $\frac{1}{m}$.

Equivalently, writing $c_i = mb_i$, the lattice consists of nonnegative integer vectors $c = (c_0, \ldots, c_{d-1})$ with $\sum c_j = m$. These are the *compositions* of the integer m into d nonnegative parts.

Figure **??** shows this discretization on a two-dimensional toy simplex, where a continuous point is rounded to a nearby lattice point.

For the formal argument, however, one needs a specific lattice point, not merely an existence statement.

Definition 3.2 (Canonical discretization)**.** Let $\mu \in \Delta_n$ and define the cumulative masses

$$M(k) \;=\; \sum_{i<k} \mu_i, \qquad 0 \le k \le d.$$

Set

$$D(k) \;=\; \lfloor m\, M(k) \rfloor, \qquad 0 \le k \le d,$$

and define the integer composition $c = (c_0, \dots, c_{d-1})$ by

$$c_i = D(i+1) - D(i) \quad (0 \le i \le d-2), \qquad c_{d-1} = m - D(d-1).$$

The associated normalized lattice point is $w_i = c_i/m$.

Remark 3.3 (Exact formal definition)**.** The Lean definition `canonical_discretization` is applied directly to the function f. It forms the same cumulative-floor expression, but writes

$$D_f(k) \;=\; \left\lfloor m\, \frac{\sum_{i<k} \mu_i}{\sum_j \mu_j} \right\rfloor$$

in terms of the bin masses $\mu_i = \int_{I_i} f$. By Lemma **??**, the denominator is 1 once $\int f = 1$.

Proposition 3.4 (Basic properties of the canonical discretization)**.** *The composition c from Definition* **??** *satisfies*

$$\sum_{i=0}^{d-1} c_i = m, \qquad \left| \frac{c_i}{m} - \mu_i \right| \le \frac{1}{m} \quad \textit{for each } i.$$

Proof sketch. The sum identity is telescoping:

$$\sum_{i=0}^{d-2} (D(i+1) - D(i)) + m - D(d-1) = m.$$

For the coordinate error, write

$$\frac{c_i}{m} - \mu_i = \left(\frac{D(i+1)}{m} - M(i+1) \right) - \left(\frac{D(i)}{m} - M(i) \right),$$

and use the elementary floor bounds $-1 < \lfloor x \rfloor - x \le 0$. In Lean these statements are formalized by `canonical_discretization_sum_eq_m` together with the coordinatewise estimate proved in `Sidon/DiscretizationError.lean`. □

3.2. The Discretization Error Bound

Define the set of *contributing bins* for a window (ℓ, s_{lo}) by

$$\mathcal{B}_{n,\ell,s_{\mathrm{lo}}} \;=\; \{ i \in \{0, \dots, d-1\} : \exists\, j \in \{0, \dots, d-1\},\ s_{\mathrm{lo}} \le i + j \le s_{\mathrm{lo}} + \ell - 2 \}.$$

Equivalently,

$$\mathcal{B}_{n,\ell,s_{\mathrm{lo}}} = \left\{ i : \max\!\big(0, s_{\mathrm{lo}} - (d-1)\big) \le i \le \min\!\big(d-1, s_{\mathrm{lo}} + \ell - 2\big) \right\},$$

which is the content of the Lean theorem `contributing_bins_iff`.

The next estimate is the exact per-window bound used later in the main theorem.

Lemma 3.5 (Per-window discretization error)**.** *Let $f \in \mathcal{F}$ satisfy $\int f = 1$, let c be the canonical discretization from Definition* **??**, *and set $w_i = c_i/m$. For a fixed window (ℓ, s_{lo}), write*

$$W = \sum_{i \in \mathcal{B}_{n,\ell,s_{\mathrm{lo}}}} w_i.$$

Then

$$\mathrm{TV}_{n,m}(c;\ell,s_{\mathrm{lo}}) - \mathrm{TV}_n^{\mathrm{cont}}(f;\ell,s_{\mathrm{lo}}) \;\le\; \frac{4n}{\ell}\left(\frac{1}{m^2} + \frac{2W}{m}\right),$$

where

$$\mathrm{TV}_{n,m}(c;\ell,s_{\mathrm{lo}}) \;=\; \frac{1}{4n\ell}\sum_{s=s_{\mathrm{lo}}}^{s_{\mathrm{lo}}+\ell-2}\sum_{i+j=s}\left(\frac{4n}{m}c_i\right)\left(\frac{4n}{m}c_j\right),$$

and $\mathrm{TV}_n^{\mathrm{cont}}(f;\ell,s_{\mathrm{lo}})$ is the corresponding continuous expression from Lemma 2.3.

Proof sketch. Write $w_i = \mu_i + \delta_i$. Expanding the quadratic expression gives linear and quadratic error terms. The quadratic part is bounded by $\frac{4n}{\ell} \cdot \frac{1}{m^2}$ because $|\delta_i| \le \frac{1}{m}$, while the linear part is bounded by $\frac{4n}{\ell} \cdot \frac{2W}{m}$ because only the contributing bins matter in the chosen window. This is precisely the estimate proved in Lean before the theorem `dynamic_threshold_sound`. □

Corollary 3.6 (Global correction bound)**.** *For every admissible window one has*

$$\mathrm{TV}_{n,m}(c;\ell,s_{\mathrm{lo}}) - \mathrm{TV}_n^{\mathrm{cont}}(f;\ell,s_{\mathrm{lo}}) \;\le\; 2n\left(\frac{2}{m} + \frac{1}{m^2}\right),$$

because $W \le 1$ and $\ell \ge 2$.

3.3. The Refined (Dynamic) Correction

The global correction bound is convenient for estimates, but the formal proof and the certificate computation use the sharper per-window threshold.

Theorem 3.7 (Dynamic threshold soundness)**.** *Let c be the canonical discretization of a normalized function $f \in \mathcal{F}$ at parameters (n,m), and let*

$$W = \frac{1}{m}\sum_{i \in \mathcal{B}_{n,\ell,s_{\mathrm{lo}}}} c_i.$$

If, for some window (ℓ, s_{lo}) with $\ell \ge 2$,

$$\mathrm{TV}_{n,m}(c;\ell,s_{\mathrm{lo}}) \;>\; c_{\mathrm{target}} + \frac{4n}{\ell}\left(\frac{1}{m^2} + \frac{2W}{m}\right),$$

then $R(f) \ge c_{\mathrm{target}}$.

Proof. Combine Lemma 2.3 with Lemma 3.4. The first gives $R(f) \ge \mathrm{TV}_n^{\mathrm{cont}}(f;\ell,s_{\mathrm{lo}})$, and the second shows that the discrete test value exceeds the continuous one by at most the correction term displayed above. □

Remark 3.8 (Formal counterpart)**.** The Lean theorem `dynamic_threshold_sound` is the formal statement of Theorem 3.6. Its hypotheses are organized in terms of the explicit composition c, written in Lean as `canonical_discretizationfnm`, the normalized contributing-bin mass $W = m^{-1}\sum_{i \in \mathcal{B}_{n,\ell,s_{\mathrm{lo}}}} c_i$, and a single window whose discrete test value exceeds the threshold.

Remark 3.9 (Integer-coordinate form)**.** If $W_{\text{int}} = \sum_{i \in \mathcal{B}_{n,\ell,s_{\text{lo}}}} c_i$, then the threshold from Theorem 3.6 can be written equivalently as

$$c_{\text{target}} + \frac{4n}{\ell} \frac{1 + 2W_{\text{int}}}{m^2}.$$

This is the exact test-value threshold that reappears in the terminal certificate at $(n, m) = (64, 20)$.

4. The Windowed Test Value

With the discretization machinery in hand, we can describe precisely what the algorithm computes at each lattice point.

4.1. Definition

Definition 4.1. For a mass vector $a = (a_0, \ldots, a_{d-1})$ with $a_i \geq 0$ and $\sum a_i = 4n$, the *autoconvolution* is the vector of length $2d - 1$:

$$\text{conv}[s] \;=\; \sum_{\substack{i+j=s \\ 0 \leq i,j \leq d-1}} a_i\, a_j, \qquad s = 0, 1, \ldots, 2(d-1).$$

The *test value* is

$$\text{tv}(a) \;=\; \max_{\substack{2 \leq \ell \leq 2d \\ 0 \leq k \leq 2(d-1)-(\ell-2)}} \frac{1}{4n\ell} \sum_{s=k}^{k+\ell-2} \text{conv}[s].$$

In words: compute the autoconvolution, slide a window of each possible width ℓ across it, sum the $\ell - 1$ consecutive entries in each window position, normalize by $4n\ell$, and take the maximum.

Remark 4.2 (Lean indexing convention)**.** The Lean definition `max_test_value` lets the starting index s_{lo} range over all values $0, \ldots, 2d - 1$, even when the window extends past the support of conv. The extra positions only append zero entries, so this convention does not change the maximum.

Remark 4.3 (Integer composition coordinates)**.** The code stores compositions $c = (c_0, \ldots, c_{d-1}) \in \mathbb{N}^d$ with $\sum_i c_i = m$ and recovers the bin-average vector by

$$a_i \;=\; \frac{4n}{m}\, c_i.$$

All pruning formulas below are stated in these integer coordinates.

4.2. Efficient Computation via Prefix Sums

The test value can be computed efficiently using prefix sums. Let $P[s] = \sum_{k=0}^{s} \text{conv}[k]$ be the prefix sum of the autoconvolution. Then the windowed sum from s_{lo} to s_{hi} is:

$$\sum_{s=s_{\text{lo}}}^{s_{\text{hi}}} \text{conv}[s] \;=\; P[s_{\text{hi}}] - P[s_{\text{lo}} - 1] \qquad (\text{with } P[-1] = 0).$$

Remark 4.4 (Complexity)**.** Computing the autoconvolution takes $O(d^2)$ time. An FFT-based method would give $O(d \log d)$, but for the small d values used here the direct method is faster in practice. The prefix sum takes $O(d)$, and the complete window scan takes another $O(d^2)$. Thus the overall cost per lattice point is $O(d^2)$.

4.3. Reversal Symmetry

A key structural observation cuts the search space in half. If we reverse the order of the bins, the autoconvolution peak does not change — intuitively, because convolution does not care which direction we label the support.

Theorem 4.5 (Autoconvolution Reversal Symmetry)**.** *For any mass vector a,* $\mathrm{tv}(a) = \mathrm{tv}(\mathrm{rev}(a))$*, where* $\mathrm{rev}(a) = (a_{d-1}, \ldots, a_0)$.

Proof. The argument has three moving parts.

Step 1: Autoconvolution reversal. Let $a' = \mathrm{rev}(a)$, so $a'_i = a_{d-1-i}$. Then:

$$\mathrm{conv}'[s] \;=\; \sum_{i+j=s} a'_i a'_j \;=\; \sum_{i+j=s} a_{d-1-i}\, a_{d-1-j}.$$

Substituting $i' = d-1-i$, $j' = d-1-j$ (so $i' + j' = 2(d-1) - s$):

$$\mathrm{conv}'[s] \;=\; \mathrm{conv}[2(d-1) - s].$$

That is, the autoconvolution of the reversal is the reversal of the autoconvolution.

Step 2: Window-sum bijection. A window sum of width $w = \ell - 1$ starting at s_0 transforms under reversal to a window sum of the same width starting at $2(d-1) - s_0 - w + 1$:

$$\begin{aligned}\sum_{s=s_0}^{s_0+w-1} \mathrm{conv}'[s] &= \sum_{s=s_0}^{s_0+w-1} \mathrm{conv}[2(d-1) - s] \\ &= \sum_{s'=2(d-1)-s_0-w+1}^{2(d-1)-s_0} \mathrm{conv}[s'].\end{aligned}$$

This is a window of the same width w, so the multiset of all window sums at width w is identical for conv and conv′.

Step 3: Normalization preserves the maximum. The normalization $\frac{1}{4n\ell}$ depends only on ℓ, not the window position. Since the multiset of window sums is identical at each width, $\mathrm{tv}(a) = \mathrm{tv}(a')$. □

Corollary 4.6 (Canonical Filter)**.** *It suffices to check only* canonical *integer compositions c with $c \le \mathrm{rev}(c)$ lexicographically. The associated bin-average vectors have the same test value, so this halves the search space (up to palindromes).*

Figure **??** depicts the reversal orbit of a composition and the lexicographic canonicalization used to choose one representative from each non-palindromic pair.

Even after exploiting reversal symmetry, the number of integer states to check — roughly $\binom{m+d-1}{d-1}/2$ — grows rapidly with d and m. To keep the computation tractable, we need ways to rule out large swathes of the simplex without computing the full test value.

5. Pruning Strategies

5.1. Asymmetry Pruning

At first glance, the following bound might seem too crude to be useful: it ignores everything about f except how much mass sits on each half of the support. Yet in practice, this single test eliminates the vast majority of configurations at every cascade level.

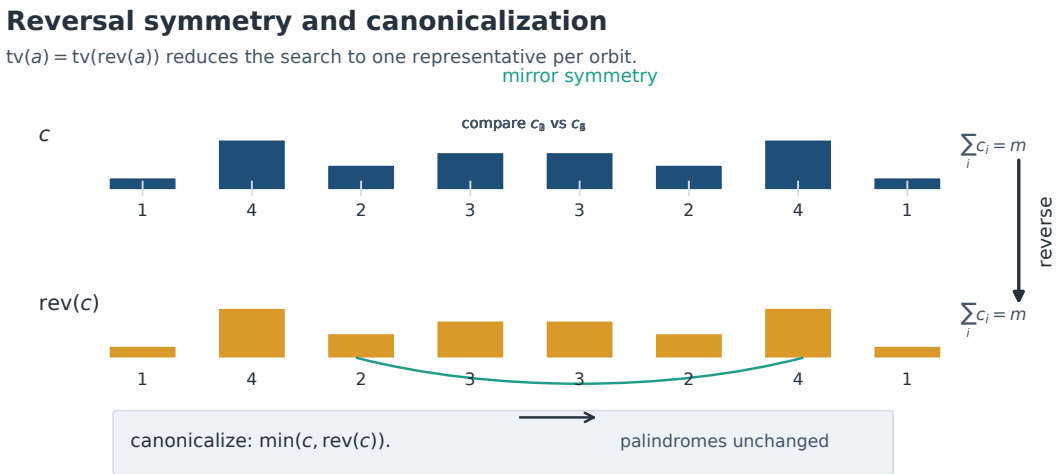

Figure 3: Reversal symmetry of the test value and the canonicalization rule $c \mapsto \min(c, \mathrm{rev}(c))$. Palindromic compositions are fixed by the involution, while non-palindromic pairs contribute only one canonical representative to the search.

Theorem 5.1 (Asymmetry Bound)**.** *Let $f \geq 0$ with $\mathrm{supp}(f) \subseteq (-\frac{1}{4}, \frac{1}{4})$ and $\int f = 1$. Let $L = \int_{-1/4}^{0} f$ be the left-half mass. Then:*

$$\|f * f\|_{L^\infty} \;\geq\; 2L^2.$$

Proof. Consider the left-left convolution:

$$(f * f)(t) \;=\; \int f(x)\, f(t-x)\, dx \;\geq\; \int_{-1/4}^{0} f(x)\, f(t-x)\, \mathbf{1}_{[-1/4,0]}(t-x)\, dx$$

for $t \in (-\frac{1}{2}, 0)$. By Fubini's theorem, the total mass of $f * f$ restricted to $(-\frac{1}{2}, 0)$ is at least L^2 (the left-left contribution). Since $|(-\frac{1}{2}, 0)| = \frac{1}{2}$:

$$\|f * f\|_{L^\infty} \;\geq\; \frac{L^2}{1/2} \;=\; 2L^2.$$

By symmetry, the same holds for the right-half mass $R = 1 - L$: $\|f * f\|_\infty \geq 2R^2$. Therefore $\|f * f\|_\infty \geq 2\max(L^2, R^2)$. □

Corollary 5.2 (Asymmetry Threshold)**.** *If $2L^2 \geq c_{\text{target}}$, i.e., $L \geq \sqrt{c_{\text{target}}/2}$, then the configuration is ruled out. The threshold left-mass fraction is:*

$$p^* \;=\; \sqrt{c_{\text{target}}/2}.$$

For $c_{\text{target}} = 1.4$: $p^* \approx 0.8367$.

Remark 5.3 (Exactness for step functions and under refinement)**.** For a step function already represented by a composition $c = (c_0, \ldots, c_{d-1})$ with $\sum c_i = m$, the left-half mass is

$$L \;=\; \sum_{i<n} \frac{c_i}{m},$$

because the midpoint $x = 0$ lies exactly on a bin boundary. This quantity is also preserved exactly under dyadic refinement: the left child bins sum to the left parent bins. Thus asymmetry pruning is an exact statement for discrete step functions inside the cascade.

Remark 5.4 (Continuous-transfer caveat)**.** The asymmetry bound is a *direct* L^∞ bound on $f * f$. It does *not* go through the discretization framework of Lemma 3.4. Accordingly, one must distinguish two roles for asymmetry:

- for an *exact step function* represented by a composition c, the criterion is immediate and needs no correction term;
- inside the certificate computation, it serves as an inexpensive early pruning rule before the more expensive window scan.

The final Lean theorem no longer depends on carrying asymmetry certificates through the continuous-to-discrete transfer. Instead, the formal theorem imports a stronger terminal-grid window certificate at $d = 128$, so asymmetry remains part of the trusted external computation but not part of the final analytic reduction.

Remark 5.5 (What the certificate computation prunes by)**.** The recorded cascade uses asymmetry pruning, the corrected window-dependent threshold, reversal canonicalization, and the single-bin cap from Section 6.3. The imported Lean axiom does not encode these intermediate pruning mechanisms; it records only the resulting terminal window witnesses.

5.2. Canonical Symmetry Exploitation

If c is an integer composition and $\tilde{c}_i = \frac{4n}{m} c_i$ is its associated bin-average vector, then Theorem 4.4 gives $\mathrm{tv}(\tilde{c}) = \mathrm{tv}(\widetilde{\mathrm{rev}(c)})$. This motivates the following notion of canonicality.

Definition 5.6. A composition c is *canonical* if $c \le \mathrm{rev}(c)$ lexicographically. That is, comparing c_0 with c_{d-1}, then c_1 with c_{d-2}, etc., c comes first (or they are equal, i.e., c is a palindrome).

Proposition 5.7 (Canonical Count)**.** *For integer compositions of a fixed total mass M into d parts, the number of canonical compositions is $\frac{N+P}{2}$, where $N = \binom{M+d-1}{d-1}$ is the total count and P is the number of palindromes.*

Proof. Every composition falls into exactly one of three categories: palindromes (those equal to their own reversal), strictly canonical non-palindromes ($b < \mathrm{rev}(b)$), and their reversals. Since reversal is an involution with no fixed points outside the palindromes, the second and third categories have equal size C. Counting gives $N = P + 2C$, so the canonical count is $P + C = (N + P)/2$. □

Remark 5.8 (Loop-bound tightening for $d = 4$)**.** For $b = (c_0, c_1, c_2, c_3)$ with total mass M and $c_3 = M - c_0 - c_1 - c_2$, canonicality ($b \le \mathrm{rev}(b)$) requires $c_0 \le c_3$, giving three loop bounds:

- $c_0 \le \lfloor M/2 \rfloor$ (if $c_0 > M/2$ then $c_3 < c_0$).
- $c_1 \le M - 2c_0$ (need $c_2 \ge 0$ with $c_1 + c_2 \le M - 2c_0$).
- $c_2 \le M - 2c_0 - c_1$ (ensures $c_3 \ge c_0$).

Plus a palindrome filter: when $c_0 = c_3$, require $c_1 \le c_2$.

For $d = 6$, analogous bounds apply: $c_0 \le \lfloor M/2 \rfloor$, $c_4 \le M - c_0 - c_1 - c_2 - c_3 - c_0$, with equality checks for the inner pairs when outer pairs tie.

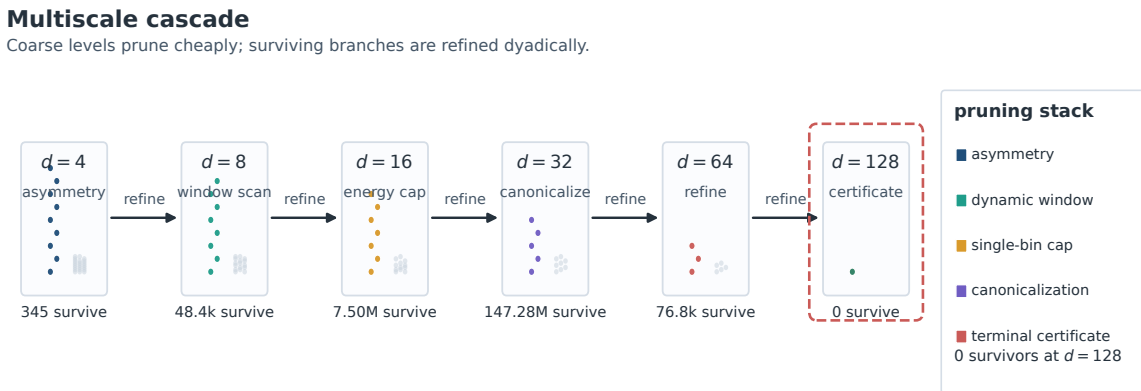

Figure 4: Stylized overview of the multiscale cascade. The annotations record the exact survivor counts from the terminal certificate run: 345 at $d = 4$, then 48,443, 7,499,382, 147,279,894, 76,829, and finally 0 at $d = 128$.

6. The Multiscale Cascade

Brute-force enumeration at fine resolution is hopelessly infeasible — even modest parameters yield billions of lattice points. The key idea is to start coarse: most configurations can be ruled out cheaply at low resolution, and only the stubborn survivors need to be examined more closely.

6.1. Overview

Step 1: Start coarse. At level L_0, enumerate all canonical compositions of m into $d_0 = 2n_0$ parts (e.g., $n_0 = 2$, $d_0 = 4$).

Step 2: Prune. For each composition, apply asymmetry pruning and then scan windows using the corrected dynamic threshold. If some window certifies the target bound, the composition is eliminated.

Step 3: Refine survivors. Each surviving composition at resolution d is refined to resolution $2d$ by *dyadic splitting*: each bin is split into two sub-bins.

Step 4: Repeat. Prune the refined compositions. Continue until either all are eliminated (proof complete) or a computational budget is reached.

Figure **??** summarizes the archived certificate run `data/cpu_cascade_20260319_201644.json`, including the exact survivor counts at each dyadic level. The repository also contains later run artefacts, notably `data/cpu_cascade_20260325_162306.json` and `data/checkpoint_meta.json`; these record subsequent implementation runs and resume metadata, not the terminal certificate quoted in Theorem 6.3.

6.2. Dyadic Splitting (Refinement)

Given a parent composition $b = (b_0, \ldots, b_{d-1})$ at resolution d, the refinement produces children $c = (c_0, c_1, \ldots, c_{2d-1})$ at resolution $2d$, where:

$$c_{2i} + c_{2i+1} = b_i \qquad \text{for each } i = 0, \ldots, d-1,$$

with $c_{2i}, c_{2i+1} \geq 0$ and both being nonnegative integers.

6.3. Energy Cap

One might worry that generating all possible children is wasteful. Indeed, many children are immediately ruled out by the single-bin diagonal window. The energy cap avoids generating them in the first place.

Recall that $a_j = \frac{4n_{\text{child}}}{m} c_j$ and the $\ell = 2$ diagonal window at position $2j$ gives

$$\text{tv}(a) \;\geq\; d_{\text{child}} \frac{c_j^2}{m^2}, \qquad d_{\text{child}} = 2n_{\text{child}}.$$

This yields the test-value cap

$$x_{\text{cap}}^{\text{TV}} \;=\; \left\lfloor m\sqrt{\frac{c_{\text{target}} + 2n_{\text{child}}\left(\frac{2}{m} + \frac{1}{m^2}\right) + 10^{-9}}{d_{\text{child}}}} \right\rfloor.$$

The implementation also applies the sharper direct cap

$$x_{\text{cap}}^{\text{CS}} \;=\; \left\lfloor m\sqrt{\frac{c_{\text{target}}}{d_{\text{child}}}} \right\rfloor,$$

and uses $x_{\text{cap}} = \min\left(x_{\text{cap}}^{\text{TV}}, x_{\text{cap}}^{\text{CS}}\right)$.

Proposition 6.1 (Energy Cap Soundness)**.** *If a child has some bin $c_j > x_{\text{cap}}^{\text{CS}}$, then any nonnegative function with those child-bin masses already satisfies $\|f * f\|_{L^\infty} > c_{\text{target}}$.*

Proof. Let $M_j = c_j/m$ be the mass in the jth child bin. Restricting f to that single bin produces a nonnegative function f_j supported on an interval of length $1/d_{\text{child}}$ and with total mass M_j. By the same averaging argument used for asymmetry pruning,

$$\|f * f\|_{L^\infty} \;\geq\; \|f_j * f_j\|_{L^\infty} \;\geq\; \frac{M_j^2}{1/d_{\text{child}}} \;=\; d_{\text{child}} \frac{c_j^2}{m^2}.$$

If $c_j > m\sqrt{c_{\text{target}}/d_{\text{child}}}$, the right-hand side exceeds c_{target}, which proves the claim. □

Remark 6.2 (Tighter Cauchy–Schwarz cap)**.** The looser cap $x_{\text{cap}}^{\text{TV}}$ mirrors the older global correction narrative, while the direct cap $x_{\text{cap}}^{\text{CS}}$ is the effective one in the implementation. It is tighter and does not need any discretization correction term.

6.4. Terminal Certificate

Theorem 6.3 (Recorded Terminal-Grid Certificate)**.** *For every composition $c = (c_0, \ldots, c_{127}) \in \mathbb{N}^{128}$ with $\sum_{i=0}^{127} c_i = 20$, let*

$$a_i \;=\; \frac{4 \cdot 64}{20} c_i, \qquad i = 0, \ldots, 127,$$

and let conv_a be the discrete autoconvolution of a. Then there exist integers $\ell \geq 2$ and s_{lo} such that

$$\frac{1}{4 \cdot 64 \cdot \ell} \sum_{s=s_{\text{lo}}}^{s_{\text{lo}}+\ell-2} \text{conv}_a[s] \;>\; \frac{7}{5} + \frac{4 \cdot 64}{\ell}\left(\frac{1}{20^2} + \frac{2}{20^2} \sum_{i \in \mathcal{B}_{64,\ell,s_{\text{lo}}}} c_i\right),$$

where $\mathcal{B}_{64,\ell,s_{\text{lo}}}$ is the set of contributing bins for that window.

Status. This is the finite computational statement imported into Lean as the axiom `cascade_all_pruned` in `Sidon/FinalResult.lean`. The archived cascade run `data/cpu_cascade_20260319_201644.json` records that the dyadic search from $d = 4$ to $d = 128$ terminates with zero survivors at level L_5. The

completeness properties discussed in Section 8 explain why that terminal outcome implies the universal quantification appearing above. The intermediate cascade levels matter only as an efficient route to this terminal statement; the final Lean theorem depends only on the quantification over all compositions in $\mathbb{N}^{128}$. Later repository files `data/cpu_cascade_20260325_162306.json` (a lower-target run) and `data/checkpoint_meta.json` (checkpoint metadata for a resumable target-$\frac{7}{5}$ search) are therefore supplementary implementation records rather than replacements for the archived certificate. □

6.5. Canonical Handling Across Levels

At level L_0, only canonical compositions are generated. At levels L_1 and beyond, the situation is more subtle:

- Children of a canonical parent P are *not* necessarily canonical.
- The reversal $\mathrm{rev}(P)$ is *not* in the parent list (unless P is a palindrome), so its children are not generated directly.
- However, if C is a child of P, then $\mathrm{rev}(C)$ is a child of $\mathrm{rev}(P)$. Since $\mathrm{tv}(C) = \mathrm{tv}(\mathrm{rev}(C))$ (Theorem 4.4), testing C suffices for both.

Therefore at levels ≥ 1:

1. Generate *all* children of each parent (no canonical filter during generation).
2. Test each child against the dynamic threshold.
3. *Canonicalize* survivors: replace each with $\min(C, \mathrm{rev}(C))$ lexicographically.
4. *Deduplicate*: remove duplicate canonical forms (children from different parents, or a palindrome parent producing both C and $\mathrm{rev}(C)$).

Proposition 6.4 (Completeness of Canonical Handling)**.** *This procedure tests every composition at the child resolution: if C' is any child-level composition and $P' = \mathit{parent}(C')$, then either P' or $\mathrm{rev}(P')$ is in the parent list, and C' or $\mathrm{rev}(C')$ is among the tested children.*

6.6. Gray-Code Fused Kernel

At deep cascade levels, each parent may have millions of children. The current CPU kernel enumerates the same child set as the naive Cartesian-product generator, but does so in mixed-radix Gray-code order and fuses generation, pruning, canonicalization, and survivor buffering into a single pass. Consecutive children differ in exactly one split coordinate, so the autoconvolution and tracked window mass can be updated locally instead of recomputed from scratch:

- **Incremental updates:** every Gray-code step changes one parent split, giving an $O(d)$ raw-convolution update and an $O(1)$ update of the tracked quick-check window mass.
- **Quick-check and ordered scans:** the previous killing window is retried first; if that fails, windows are scanned in a profile-guided order, but every admissible (ℓ, s_{lo}) is still checked before a child is declared a survivor.
- **Lookup-table thresholds and staged survivor writes:** the conservative integer threshold is precomputed as a table in (ℓ, W_{int}), and surviving rows are batched through an L1-resident staging buffer before they are flushed to the main output array.
- **Block skipping:** subtree pruning uses partial autoconvolutions of fixed prefixes, and the innermost Gray-code sweep can be fast-forwarded when a univariate range check proves that the same killing window eliminates the rest of that sweep.

7. The Complete Proof

7.1. Algorithm

Algorithm 1 Cascade Search For Terminal Coverage

Require: c_{target}, grid resolution m, starting half-bins n_0, max levels $L_{\max}$

```
d_0 ← 2n_0
Generate all canonical compositions of m into d_0 parts          ▷ integer composition coordinates
Apply asymmetry pruning (threshold √(c_target/2))
For every remaining composition, scan all windows and prune if some window exceeds the dynamic
threshold from Theorem 3.6
survivors_0 ← unpruned compositions
for k = 1, 2, ..., L_max do
    d_k ← 2d_{k−1};   n_k ← 2n_{k−1}
    Compute the single-bin child cap x_cap = min(x_cap^TV, x_cap^CS)
    for all parent P ∈ survivors_{k−1} do
        for all children C of P in mixed-radix Gray-code order (dyadic split with energy cap) do
            Apply asymmetry pruning to C
            if not asymmetry-pruned then
                Re-try the previous killing window; if needed, scan all windows of C in profile-guided
order using the lookup-table threshold
                if some window exceeds the dynamic threshold then
                    Prune C
                else
                    C survives; canonicalize and store
                end if
            end if
        end for
    end for
    Deduplicate survivors (canonicalize → sort → scan)
    survivors_k ← deduplicated survivors
    if |survivors_k| = 0 then
        return ZERO-SURVIVORS
    end if
end for
return INCONCLUSIVE: |survivors_{L_max}| compositions remain
```

7.2. Main Theorem

Theorem 7.1 (Main theorem)**.** *Let* $f : \mathbb{R} \to \mathbb{R}$ *be nonnegative, satisfy* $\operatorname{supp}(f) \subseteq (-\frac{1}{4}, \frac{1}{4})$*, and obey* $\int f > 0$*. If* $\|f * f\|_{L^\infty} < \infty$*, then*

$$\frac{\|f * f\|_{L^\infty}}{\left(\int f\right)^2} \geq \frac{7}{5}.$$

Consequently $C_{1a} \geq \frac{7}{5} = 1.4$.

Remark 7.2 (Exact Lean statement)**.** The Lean theorem `autoconvolution_ratio_ge_7_5` is phrased for the function `autoconvolution_ratio` and carries an explicit finiteness hypothesis `h_conv_fin` asserting

that the essential supremum of $f * f$ is not $\top$. This is the formal counterpart of the hypothesis $\|f * f\|_{L^\infty} < \infty$ in Theorem 7.1.

Proof. Normalize

$$g = \frac{f}{\int f},$$

so that $\int g = 1$. The autoconvolution ratio is scale-invariant, so it suffices to prove $R(g) \geq \frac{7}{5}$.

At parameters $(n, m) = (64, 20)$, the identity `sum_bin_masses_eq_one` gives total bin mass 1, so the cumulative-floor construction `canonical_discretization` produces a composition $c \in \mathbb{N}^{128}$ with $\sum_i c_i = 20$. By the terminal certificate Theorem 6.3, there exist (ℓ, s_{lo}) such that

$$\mathrm{TV}_{64,20}(c; \ell, s_{\text{lo}}) > \frac{7}{5} + \frac{4 \cdot 64}{\ell}\left(\frac{1}{20^2} + \frac{2}{20^2} \sum_{i \in \mathcal{B}_{64,\ell,s_{\text{lo}}}} c_i\right).$$

This is exactly the hypothesis needed for Theorem 3.6, which in Lean is the theorem `dynamic_threshold_sound`. Its proof combines the continuous lower bound `continuous_test_value_le_ratio` with the per-window discretization estimate from Lemma 3.4. Therefore $R(g) \geq \frac{7}{5}$, and scale invariance gives $R(f) \geq \frac{7}{5}$.

Taking the infimum over $f \in \mathcal{F}$ yields $C_{1a} \geq \frac{7}{5}$. □

8. Correctness Invariants

The external computation matters only through the finite statement of Theorem 6.3. To justify importing that statement, one must know that the search implementation really tested every relevant terminal composition against the correct threshold. The following invariants isolate the required claims.

8.1. Invariant 1: Complete Enumeration

Every integer composition $c \in \mathbb{N}^d$ with $\sum_i c_i = m$ must be either directly checked or shown to be covered by a checked ancestor and all its refinements.

- At L_0: all $\binom{m+d_0-1}{d_0-1}$ compositions are generated, up to the reversal symmetry from Section 4.3.
- At L_k $(k \geq 1)$: all children of all surviving parents are generated, with no sampling or probabilistic filtering.

8.2. Invariant 2: All Windows Checked

For each composition that is not eliminated by asymmetry, every valid window (ℓ, k) with $2 \leq \ell \leq 2d$ and all valid starting positions k must be tested before the composition is declared a survivor. The scan order may be profile-guided and may begin with a quick-check window carried over from the previous child, but early exit is permitted only after a pruning window has been found.

8.3. Invariant 3: Conservative Arithmetic

Unsound threshold arithmetic would invalidate the proof. The computation avoids this as follows:

- Integer coordinates: the autoconvolution is computed in exact integer arithmetic.
- Conservative threshold: the code evaluates the corrected integer threshold

$$c_{\text{target}} m^2 \frac{\ell}{4n} + 1 + 2W_{\text{int}},$$

  together with an added $10^{-9} m^2$ margin and a final $(1 - 4\epsilon_{\text{mach}})$ factor before flooring.

- The multiplicative factor lowers the threshold slightly, but the additive $10^{-9}m^2$ margin is overwhelmingly larger, so the net effect remains conservative: the code may miss a borderline prune, but it does not falsely prune.

8.4. Invariant 4: Refinement Completeness

Every refinement of a surviving parent must be checked at the finer level. The dyadic splitting with energy cap generates all valid children (Section 6.3), and children exceeding the energy cap are provably pruned.

8.5. Invariant 5: Optimization Preservation

The current implementation layers several performance-only transformations on top of the logical search: mixed-radix Gray-code enumeration, lookup-table threshold evaluation, staged survivor writes, subtree pruning, and univariate sweep skipping. For the proof, these optimizations are admissible only if they preserve two things:

- the set of children that are logically visited at each level;
- the prune/survive verdict associated to each visited child.

The repository therefore keeps these obligations separate from the core continuous-to-discrete reduction: the analytic theorem depends only on the terminal certificate, while the implementation audit tracks why the optimized kernel still computes that certificate correctly.

9. Computational Verification

The implementation was audited at four levels: enumeration, test-value evaluation, implementation-preserving kernel optimizations, and terminal certification.

9.1. Composition Generators

- Generator output counts were checked against the stars-and-bars formula $\binom{m+d-1}{d-1}$ for representative parameter choices, confirming completeness, uniqueness, and the sum constraint.
- Specialized low-dimensional generators were checked against a generic reference generator.
- Canonical enumeration was checked against the exact count $\frac{N+P}{2}$ from Proposition 5.7.

9.2. Test Value Computation

- Reversal symmetry was verified on exhaustive small instances and additional random tests.
- An independent reference implementation agrees with the optimized kernel on all canonical compositions at the coarse test level $d = 4$, $m = 20$.

9.3. Pruning Correctness

- Exhaustive coarse-level checks confirm that the window test never prunes a composition whose exact test value lies below the dynamic threshold.
- The same checks confirm that every composition whose exact test value exceeds the threshold is indeed eliminated.
- The asymmetry filter was checked directly on exact step functions and is symmetric under reversal.

9.4. Implementation-Preserving Kernel Optimizations

- The mixed-radix Gray-code kernel was checked against the earlier lexicographic fused kernel on representative parents across multiple child dimensions, confirming identical canonical survivor sets and complete child coverage.

- The threshold lookup table is only a cache of the same conservative integer comparison used by the direct per-window computation, so it changes evaluation cost rather than the pruning predicate.
- The staging buffer, subtree-pruning fast-forward, and univariate sweep-skip mechanisms are treated as separate audit items: each must preserve the survivor stream and may skip blocks only when those blocks are already certified prunable by a sound comparison.

9.5. Formula Cross-Check

The integer threshold used in the search is algebraically equivalent to the test-value threshold in Theorem 3.6. Relative to the exact mathematical threshold, the implementation adds a small positive safety margin before flooring, so the pruning rule is conservative.

9.6. Final Cascade Run

The recorded CPU cascade in `data/cpu_cascade_20260319_201644.json` reaches zero survivors at level L_5, corresponding to dyadic refinement from $d = 4$ through $d = 128$. The companion log `data/cpu_job.log` records the same terminal outcome after roughly 70.08 hours of wall-clock time. In the accompanying Lean formalization, this finite computation is packaged as the axiom `cascade_all_pruned`, which states directly that every terminal composition at $d = 128$ (equivalently, $n = 64$) with total mass $m = 20$ has some window whose test value exceeds the corrected dynamic threshold at target $\frac{7}{5}$. The newer files `data/cpu_cascade_20260325_162306.json` and `data/checkpoint_meta.json` are kept in the repository as later run and resume artefacts, but they are not the terminal certificate imported by Lean.

9.7. Lean Formalization Status

The accompanying formalization consists of a modular Lean 4 development under `lean/Sidon/` and a consolidated export in `lean/complete_proof.lean`. It is carried out in Lean 4 [**?**], on top of the mathlib library [**?**]. The development formalizes the reduction in separate files for definitions, step functions, continuous test values, discretization error, pruning lemmas, and the final normalization argument. The key analytic files for the present paper are `Sidon/TestValueBounds.lean`, `Sidon/DiscretizationError.lean`, and `Sidon/FinalResult.lean`. Separate implementation-audit files `Sidon/GrayCodeSubtreePruning.lean`, `Sidon/ThresholdLookupTable.lean`, `Sidon/StagingBuffer.lean`, and `Sidon/UnivariateSweepSkip.lean` isolate the proof obligations introduced by the current Gray-code CPU kernel.

Because `autoconvolution_ratio` is encoded using `ENNReal.toReal`, the formal theorem carries an explicit finiteness hypothesis on the essential supremum of $f * f$.

The development is not completely axiom-free. The final reduction rests on one imported statement:

- the computational certificate `cascade_all_pruned`, expressing the terminal-grid statement at $d = 128$.

Everything else in the analytic reduction is proved in Lean.

10. Discussion

10.1. Relationship to the Original CS17 Proof

Our framework is a direct extension of Cloninger–Steinerberger [1]. The key differences:

1. **Dynamic correction**: We use a sharper per-window correction rather than only a coarse global bound. This improves pruning while preserving rigor.
2. **Multiscale cascade**: CS17 used $m = 50$ and refined from $n = 3$ through $n = 24$. We use $m = 20$ with dyadic refinement from $d = 4$ through $d = 128$.
3. **Fused generation**: Instead of materializing all children and then testing them, we generate and test

each child inline, reducing memory from $O(N_{\text{children}} \cdot d)$ to $O(d)$ per parent.

4. **Integer arithmetic**: Autoconvolution and threshold comparisons use exact integer operations, with floating-point involved only in the conservative threshold computation.

10.2. Limitations

- The proof is *computer-assisted*: the mathematical argument reduces correctness to a finite computation, but that computation must be performed by a machine. The invariants in Section 8 ensure that the computation is trustworthy.
- The approach is limited by the exponential growth of the composition count with d and m. The multiscale cascade mitigates this, but very aggressive targets (c_{target} close to C_{1a}) will require more cascade levels and higher computational cost.
- The Lean theorem imports the terminal certificate `cascade_all_pruned` rather than replaying the full 70-hour search inside the Lean kernel. Eliminating this final axiom would require a kernel-level certification of the same finite computation.
- The formal statement isolates the case $\|f * f\|_{L^\infty} < \infty$ because of the way the ratio is encoded in Lean. This is harmless for the lower bound on C_{1a}, since functions with unbounded self-convolution already satisfy every finite lower bound vacuously.

Acknowledgments

This work builds on the foundational algorithm of Cloninger and Steinerberger [1].